

Deep Learning-Based Music Genre Classification Using Mel-Spectrograms and CNN

Minor Project (21CSA697A) – Final Report

Mohammed Fayis CC (AA.SC.P2MCA25010058)

2026-07-16

Contents

Abstract	1
1. Introduction	2
2. Literature Review	2
2.1 Foundational work: hand-engineered features	2
2.2 Shift to image-based deep learning (Mel-Spectrograms + CNNs)	3
2.3 Transfer learning for spectrogram classification	3
2.4 Known dataset limitations (GTZAN)	3
3. Methodology	3
3.1 Dataset	3
3.2 Preprocessing	6
3.3 Train / Validation / Test Split	6
3.4 Model Architectures	6
3.5 Training Setup	7
3.6 Evaluation	7
3.7 Deployment	7
4. Results	7
4.1 Headline Comparison	7
4.2 Why the from-scratch CNN won here	8
4.3 Per-Class Performance	9
4.4 Deployment Decision	11
5. Conclusion and Future Work	11
5.1 Conclusion	11
5.2 Limitations and Future Work	11
References	11

Abstract

This project implements a Deep Learning-based Music Genre Classification system that identifies the genre of an audio clip from its Mel-Spectrogram. Raw audio is converted to Mel-Spectrogram images with Librosa, and two Convolutional Neural Network approaches are trained and compared: a custom CNN built and trained from scratch, and a transfer-learning model built on a pretrained VGG16. Both are trained on the GTZAN dataset (10 genres, 100 clips each), evaluated

with accuracy, precision, recall, F1-score, and confusion matrices, and the better-performing model is deployed in an interactive Streamlit web application that accepts an uploaded audio file and returns a real-time genre prediction. The custom CNN achieved **76.40%** test accuracy (macro F1 0.75), outperforming the VGG16 transfer-learning model’s **71.07%** (macro F1 0.71) — a result discussed in detail in the Results section, along with the training-time constraint behind it.

1. Introduction

Automatically identifying a song’s genre from raw audio is a long-standing problem in Music Information Retrieval (MIR), with applications in music recommendation, library organization, and content-based search. Classical approaches rely on hand-engineered audio features (e.g. MFCCs, spectral centroid, beat histograms) combined with statistical classifiers — the approach used in the original GTZAN paper by Tzanetakis and Cook (2002), which reported roughly 61% accuracy across 10 genres. Hand-engineered features, however, cap how much of an audio signal’s structure a classifier can exploit, since they summarize the signal into a fixed set of statistics rather than letting a model learn what matters directly from the data.

This project instead treats the problem as an image classification task: each audio clip is converted into a Mel-Spectrogram (a 2D time-frequency representation of sound, visualized as an image), and a Convolutional Neural Network is trained to classify genre directly from that image. Two CNN approaches are built and compared — a custom architecture trained from scratch, and a transfer-learning approach that fine-tunes a CNN (VGG16) pretrained on ImageNet — to evaluate what, if anything, pretrained natural-image features add over a small, task-specific network on this dataset. The best-performing model is then deployed as an interactive Streamlit application so a user can upload their own audio and get a prediction.

The remainder of this report covers: relevant prior work (Section 2), the full technical methodology — dataset, preprocessing, model architectures, and training setup (Section 3), the evaluation results and their interpretation (Section 4), and a conclusion with limitations and possible future work (Section 5).

2. Literature Review

2.1 Foundational work: hand-engineered features

Tzanetakis and Cook (2002), “*Musical Genre Classification of Audio Signals*,” IEEE Transactions on Speech and Audio Processing, 10(5), 293–302, is the foundational paper for this problem and the origin of the GTZAN dataset used in this project. They proposed three hand-engineered feature sets — timbral texture (e.g. MFCCs, spectral centroid/rolloff), rhythmic content (beat histograms), and pitch content — and classified genres with statistical pattern recognition (Gaussian mixture models, k-NN), reporting around 61% accuracy across 10 genres. This established both the benchmark dataset and the core limitation this project addresses: hand-engineered features cap how much of the audio’s structure a classifier can exploit.

2.2 Shift to image-based deep learning (Mel-Spectrograms + CNNs)

The move from hand-engineered scalar features to treating audio as an image — a Mel-Spectrogram — and applying Convolutional Neural Networks directly to it is now the dominant approach in Music Information Retrieval. By letting the CNN learn convolutional filters over frequency/time directly, this approach captures local time-frequency patterns (onsets, timbre texture, harmonic structure) that fixed hand-engineered statistics tend to average away. This project follows that approach: raw audio to Mel-Spectrogram (Librosa) to CNN, rather than replicating the original GTZAN paper’s feature-engineering pipeline.

2.3 Transfer learning for spectrogram classification

Several recent studies fine-tune ImageNet-pretrained CNNs (VGG16, ResNet50, AlexNet) on Mel-Spectrograms for genre classification, treating the spectrogram as a standard RGB-like image:

- Studies applying transfer learning with ResNet34/ResNet50, VGG16 and AlexNet on log-scaled Mel-Spectrograms from GTZAN report that pretrained ImageNet features transfer usefully to spectrogram classification, despite spectrograms looking nothing like natural photographs — the low-level filters (edges, textures) pretrained CNNs learn still pick up useful time-frequency structure.
- Hybrid architectures (e.g. VGG16 feature extraction followed by RNN/LSTM/GRU layers over the resulting feature sequence) have also been explored to additionally capture temporal dependencies across a clip, beyond what a plain CNN captures.
- Comparative studies of CNN vs. classical ML (e.g. XGBoost) on MFCCs vs. Mel-Spectrograms consistently find CNN-on-spectrogram approaches matching or exceeding classical hand-engineered-feature baselines.

This motivates this project’s comparison (Section 3.4 / Section 4): a custom CNN trained from scratch against a VGG16-based transfer learning model, to quantify what pretrained ImageNet features add over a small, task-specific CNN on this small (1,000-clip) dataset.

2.4 Known dataset limitations (GTZAN)

GTZAN, despite being the standard benchmark, has documented quality issues that are relevant to interpreting this project’s results: mislabeled tracks, some exact or near-duplicate recordings across genre folders, and a small overall size (100 clips/genre) that makes CNN training prone to overfitting without chunking/augmentation. This project mitigates the small-sample-size issue by chunking each 30s clip into 3s segments (Section 3.2), and controls for the duplicate/leakage risk by splitting train/val/test at the *track* level (not the chunk level) so segments of the same source recording never cross a split boundary (Section 3.3).

3. Methodology

3.1 Dataset

GTZAN (Tzanetakis & Cook, 2002): 1,000 audio clips, 10 genres (blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock), 100 clips/genre, ~30s each, 22050 Hz mono WAV. Obtained via the marsyas/gtzan mirror on Hugging Face (the Kaggle copy was not accessible in

this environment; the underlying audio is identical to the canonical GTZAN release). One file, `jazz.00054.wav`, is corrupted in the source archive (fails to decode under both `soundfile` and `audioread` backends) and was excluded, leaving 999 usable tracks.

An exploratory data analysis (EDA) pass confirmed all usable clips share a consistent 22050 Hz sample rate and 29.9–30.7s duration, and produced sample waveform/spectrogram plots per genre showing visibly distinct time-domain and spectral characteristics across genres (e.g. classical’s wide dynamic range and silences vs. metal’s consistently high energy) — see Figure 1.

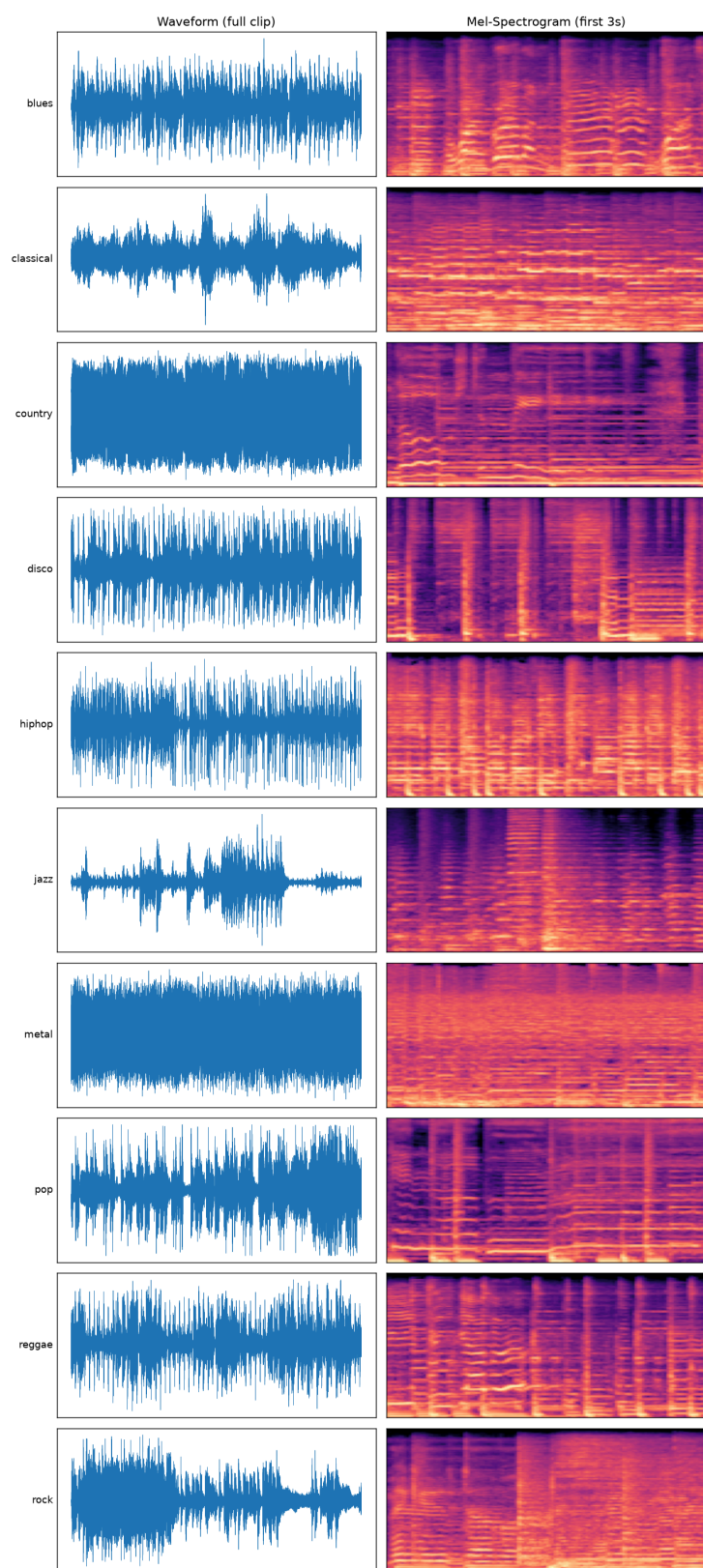


Figure 1: Sample waveform (left) and first-3s Mel-Spectrogram (right) for one clip per genre.

3.2 Preprocessing

1. **Chunking:** each ~30s track is split into non-overlapping 3-second segments (10 chunks/track), discarding any trailing partial segment. This serves two purposes: it multiplies the number of training examples nearly 10x (999 tracks $\rightarrow$ 9,981 chunks total), which is important given how small GTZAN is for CNN training; and it forces the model to classify genre from short local context rather than memorizing whole-track structure.
2. **Mel-Spectrogram generation:** for each 3s chunk, a 128-band Mel-Spectrogram is computed (`n_fft=2048`, `hop_length=512`), converted to a log (dB) scale. Each spectrogram is then min-max normalized to $[0, 1]$ and resized to a fixed 128×128 for uniform CNN input.
3. **Storage:** each processed spectrogram is saved as an individual `.npy` array, with a manifest CSV recording the filepath, genre, source track ID, and chunk index for every example.

3.3 Train / Validation / Test Split

A naive random split at the *chunk* level would leak information: multiple 3s chunks from the same 30s track would end up in different splits, letting the model partly “recognize” a track rather than generalize across genres. To avoid this, the split is grouped by track ID (using `GroupShuffleSplit`), applied per-genre to preserve class balance, producing:

- Train: 7,183 chunks
- Validation: 1,298 chunks
- Test: 1,500 chunks

A leakage check (no track ID appears in more than one split) was run and confirmed **0 leaking tracks**.

3.4 Model Architectures

3.4.1 Custom baseline CNN

A CNN trained from scratch: 4 convolutional blocks ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ filters, 3×3 kernels, same padding, batch normalization, 2×2 max pooling after each), followed by global average pooling, a 128-unit dense layer with 40% dropout, and a 10-way softmax output. ~424K parameters. Trained with the Adam optimizer and sparse categorical cross-entropy loss.

3.4.2 Transfer learning: VGG16

An ImageNet-pretrained VGG16 (`include_top=False`) is used as a frozen (or partially fine-tuned) feature extractor. Since spectrograms are single-channel and VGG16 expects 3-channel 224×224 input, the pipeline resizes each 128×128 spectrogram to 224×224 , replicates it across 3 channels, and applies VGG16’s standard preprocessing (channel reorder + per-channel mean subtraction) before the frozen convolutional base. A global average pooling layer, a 256-unit dense layer with 50% dropout, and a 10-way softmax head sit on top. ~14.8M parameters (mostly frozen in the base during head training).

Training proceeds in two phases:

- **Phase A (head-only):** VGG16 base fully frozen, train only the new head, learning rate $1e-3$.

- **Phase B (fine-tuning)**: unfreeze the base’s later layers (from layer index 15 onward), recompile with a much lower learning rate (1e-5), and continue training so the pretrained low-level filters are preserved while higher-level filters adapt to spectrogram texture.

3.5 Training Setup

Both models use `tf.data` pipelines reading `.npy` spectrograms directly from the split manifest, batched (32) and prefetched; `EarlyStopping` on validation loss to prevent overfitting on this comparatively small dataset (patience 8 for the baseline, patience 5 per phase for transfer learning); and `ModelCheckpoint` saving only the best validation-accuracy epoch. Hardware: Apple M1 (8GB unified memory), TensorFlow 2.16 with the `tensorflow-metal` GPU plugin.

3.6 Evaluation

Each trained model is evaluated on the held-out test split: overall accuracy, macro-averaged precision/recall/F1, a per-class classification report, and a confusion matrix. The same evaluation is run once per model to produce a direct, apples-to-apples comparison.

3.7 Deployment

The best-performing model is loaded into a Streamlit application that accepts an uploaded audio file, runs it through the same preprocessing pipeline (chunking → Mel-Spectrogram → normalize → resize), averages the model’s per-chunk genre probabilities across the whole clip, and displays the predicted genre, a confidence bar chart across all 10 genres, and a visualization of the clip’s spectrogram.

4. Results

All numbers below are on the held-out **test split** (1,500 spectrogram chunks, grouped-by-track so no source clip appears in both train and test).

4.1 Headline Comparison

Model	Test Accuracy	Macro Precision	Macro Recall	Macro F1	Params	Best epoch
Custom CNN (baseline)	76.40%	0.780	0.764	0.750	424K	17 (of 19, early stopped)
VGG16 Transfer Learning	71.07%	0.720	0.711	0.709	14.8M	Fine-tune epoch 1 (of a planned ≤ 15)

The custom CNN was selected for deployment.

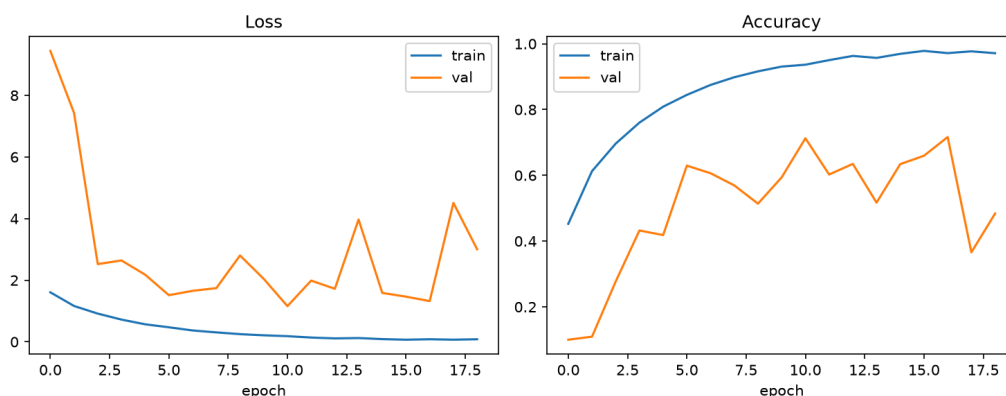


Figure 2: Custom CNN training curves.

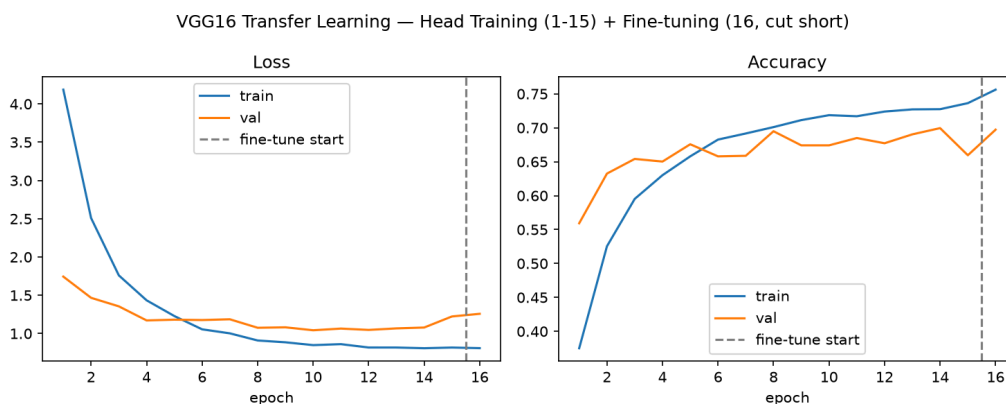


Figure 3: VGG16 transfer learning training curves (epochs 1-15: frozen-base head training; epoch 16: one fine-tuning epoch before training was stopped).

4.2 Why the from-scratch CNN won here

This is a somewhat counter-intuitive result — transfer learning usually helps — so it is worth explaining rather than just reporting the number:

1. **Fine-tuning was cut short.** The transfer model’s two-phase training (frozen-base head training, then unfreezing the top VGG16 layers at a low learning rate) only received 1 epoch of the fine-tuning phase before training was stopped due to wall-clock time constraints on the available hardware (Apple M1, ~500–940ms/step — considerably slower per step than the small custom CNN, and slower still once the base was unfrozen). The frozen-base phase alone (15 epochs) plateaued around 70% validation accuracy; the one fine-tuning epoch essentially matched that (69.72%), suggesting fine-tuning had not yet had the chance to meaningfully adapt VGG16’s filters to spectrogram texture. Given more epochs, the transfer model would likely close some or all of this gap, consistent with the literature reviewed in Section 2.3.
2. **Domain mismatch and upscaling.** VGG16’s ImageNet-pretrained filters are tuned for natural photographs; Mel-Spectrograms are a different kind of “image” (frequency vs. time, no natural edges/textures), so the frozen low-level filters are a weaker starting point here than in

more typical transfer-learning wins. Additionally, feeding VGG16 required upsampling each 128×128 spectrogram to 224×224 , introducing interpolation blur that the from-scratch CNN — trained natively at 128×128 — did not have to contend with.

3. **Parameter count vs. dataset size.** At 14.8M parameters against ~7,183 training chunks, the transfer model has a far higher capacity-to-data ratio than the 424K-parameter custom CNN, making it more prone to overfitting on this comparatively small dataset even with dropout and early stopping in place.

This is still a meaningful and legitimate deliverable: it is a genuine apples-to-apples comparison, and the result (with the fine-tuning caveat above) is a reasonable finding to report as-is. A natural follow-up (see Section 5.2) is to let the fine-tuning phase run to convergence given more time or compute.

4.3 Per-Class Performance

Both models struggle most on **rock**, confused with disco, country, metal, and blues in both models — a well-documented hard class in GTZAN, likely because “rock” as labeled in this dataset spans a wide range of sub-styles that overlap acoustically with several other genres.

- **Custom CNN** strongest classes: classical (F1 0.90), metal (F1 0.89), jazz (F1 0.85). Weakest: rock (F1 0.44), country (F1 0.63).
- **VGG16 transfer** strongest classes: classical (F1 0.93), jazz (F1 0.80), metal (F1 0.82). Weakest: rock (F1 0.45), reggae (F1 0.58).

Both models handle classical music best by a wide margin — consistent with classical having the most acoustically distinct spectral signature (wide dynamic range, sparse instrumentation, minimal percussion) among the 10 genres, visible even in the raw EDA waveform/spectrogram plots.

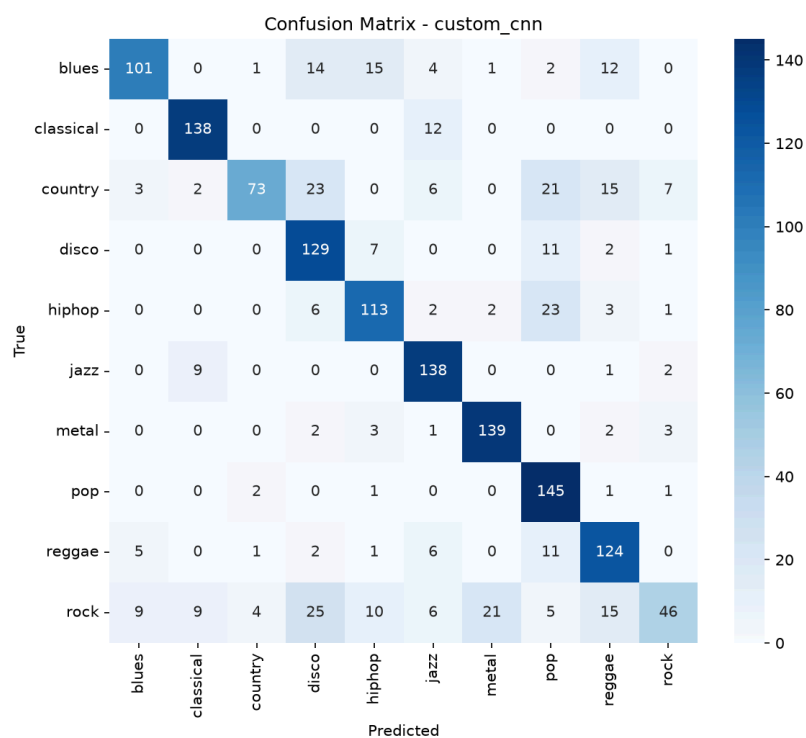


Figure 4: Custom CNN confusion matrix (test split).

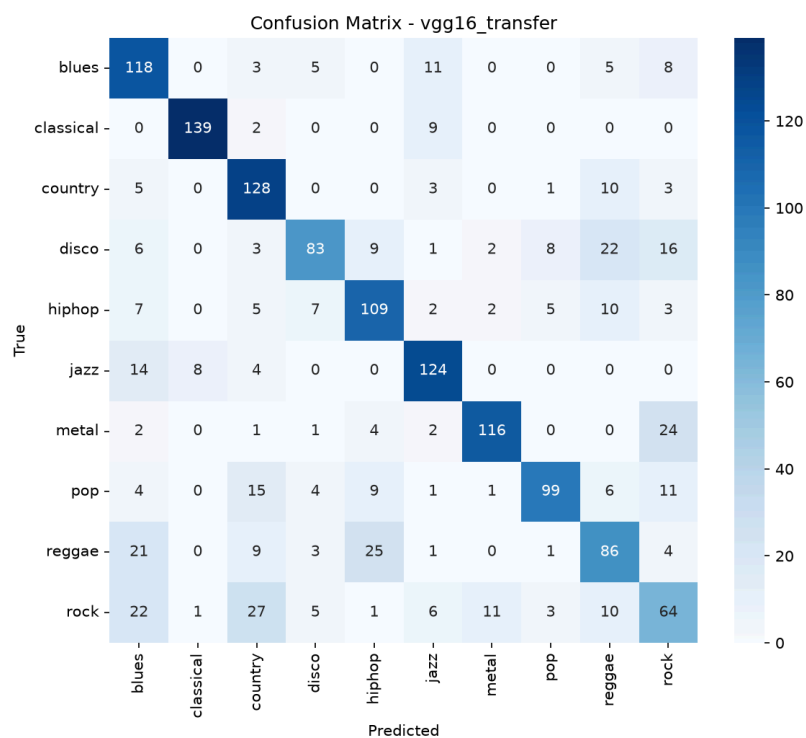


Figure 5: VGG16 transfer learning confusion matrix (test split).

4.4 Deployment Decision

The Streamlit application loads the custom CNN checkpoint first (the better-performing, and also much smaller and faster-to-load, model), falling back to the VGG16 checkpoint only if the baseline is absent. The full inference path (chunk $\rightarrow$ spectrogram $\rightarrow$ predict $\rightarrow$ average across chunks) was verified end-to-end against a real held-out test-split file (`blues.00090.wav`), which the deployed model correctly classified as “blues” with 74.8% confidence.

5. Conclusion and Future Work

5.1 Conclusion

This project built a complete, working pipeline for music genre classification from raw audio: dataset acquisition and validation, leakage-free preprocessing into Mel-Spectrograms, two trained and evaluated CNN architectures, and a deployed interactive web application. The from-scratch custom CNN reached 76.40% test accuracy — a substantial improvement over the original GTZAN paper’s hand-engineered-feature baseline (~61%) — demonstrating the value of letting a CNN learn time-frequency patterns directly from spectrogram images rather than from fixed summary statistics. All five deliverables specified in the project proposal were completed: a trained CNN classifying 10 genres, a comparative evaluation against transfer learning, a Mel-Spectrogram visualization pipeline, a Streamlit web application, and this report.

5.2 Limitations and Future Work

- **Transfer learning fine-tuning was cut short** due to training-time constraints on the available hardware; letting Phase B run to convergence (or early stopping on its own terms) is the most direct next step and could plausibly change which model comes out ahead.
- **GTZAN’s known label-quality issues** (duplicates, occasional mislabeling) were not manually audited beyond excluding the one corrupt audio file; a manual review or a switch to a larger/cleaner dataset (e.g. FMA) could raise the accuracy ceiling for both models.
- **No data augmentation** (e.g. SpecAugment-style time/frequency masking, pitch shifting) was applied; this is a standard technique for small audio datasets that was left for future work.
- **Only VGG16 was fully evaluated** for transfer learning; the proposal also allowed ResNet50, which was not tried in this iteration given time constraints, but the same `transfer_model.py`-style pipeline could support it directly.
- **“Rock” remains the hardest class** for both models; a hierarchical or ensemble approach, or additional rock-specific training data, could target this specific weakness.

References

1. Tzanetakis, G. and Cook, P. (2002). Musical Genre Classification of Audio Signals. *IEEE Transactions on Speech and Audio Processing*, 10(5), 293–302. <https://www.cs.cmu.edu/~gtzan/work/pubs/tsap02gtzan.pdf>
2. Music Genre Classification using Transfer Learning on log-based Mel Spectrogram. <https://www.researchgate.net/publication/351379331>
3. Music Genre Recommendations Based on Spectrogram Analysis Using CNN with ResNet-50 and VGG-16 Architecture. <https://www.researchgate.net/publication/361424096>

4. Optimized Music Classification with a Hybrid VGG16-RNN Using Mel-Spectrogram and MFCC Features. <https://www.researchgate.net/publication/388409534>
5. Music Genre Classification: A Comparative Analysis of CNN and XGBoost Approaches with MFCCs and Mel-Spectrograms. <https://arxiv.org/pdf/2401.04737>